4

Stacks

4.1 INTRODUCTION

A *stack* is a linear data structure and very much useful in various applications of computer science. The implementation of the majority of systems programs is simplified using this data structure. Before discussing this data structure, let us first consider a few examples of the stack phenomenon.

Shunting of trains in a railway yard

Suppose there is a railway yard with a single track. Trains enter into the railway yard for placement, and when they exit, it is just in opposite order to that they had entered, i.e. the last train comes out first (see Figure 4.1).

Shipment in a cargo

For the shipment of goods, they are loaded into a cargo compartment. At the destination, they are unloaded exactly in the opposite sequence to that in which they were loaded. That is, the goods loaded last get unloaded first.

Plates on a tray

Suppose a chef placed the dishes on a tray one above the other. The waiter served the dishes to the customers in the opposite order that the chef placed them, that is, the dish at the top which

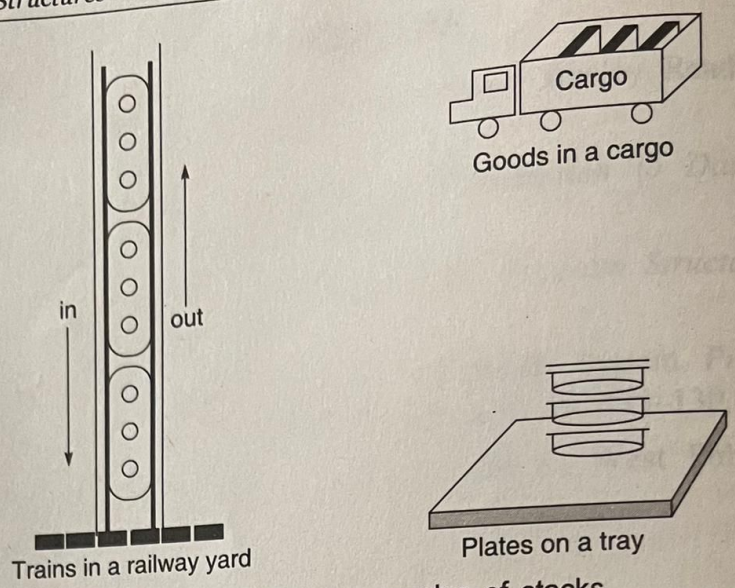


Figure 4.1 Some examples of stacks.

was placed last by the chef is serviced first. The first dish placed by the chef on the tray is serviced last by the waiter.

From the above examples, it is clear that a *stack* is something which follows the last-in first-out strategy. This is why a stack is alternatively termed LIFO (Last-In-First-Out).

4.2 DEFINITION

A *stack* is an ordered collection of homogeneous data elements where the insertion and deletion operations take place at one end only.

Like an array and a linked list, a stack is also a linear data structure but the only difference is that in the case of the former two, insertion and a deletion operations can take place at any position. The insertion and deletion operations in the case of a stack are specially termed PUSH and POP, respectively, and the position of the stack where these operations are performed is known as the TOP of the stack. An element in a stack is termed an ITEM. The maximum number of elements that a stack can accommodate is termed SIZE. Figure 4.2 shows a typical view of a stack data structure.

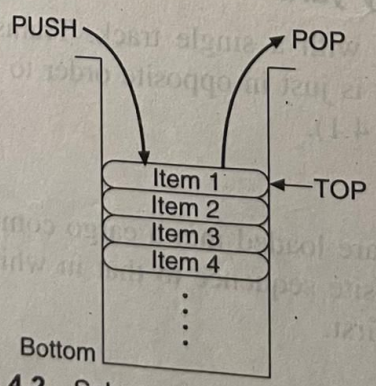


Figure 4.2 Schematic diagram of a stack.

4.3 REPRESENTATION OF A STACK

A stack may be represented in the memory in various ways. There are two main ways: using a one-dimensional array and a single linked list. Representations of stacks in a memory are discussed in the following two sections.

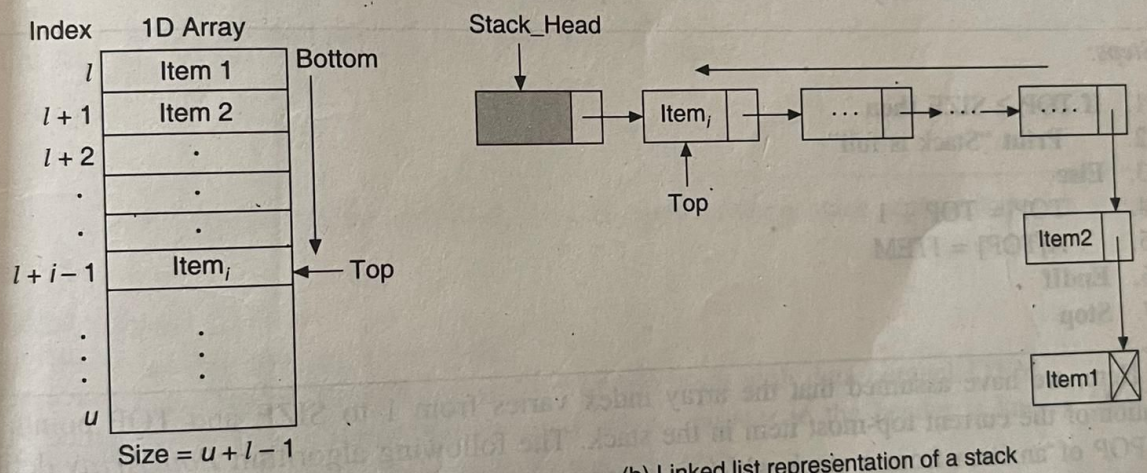
4.3.1 Array Representation of Stacks

First we have to allocate a memory block of sufficient size to accommodate the full capacity of the stack. Then, starting from the first location of the memory block, the items of the stack can be stored in a sequential fashion.

In Figure 4.3(a), $ITEM_i$ denotes the ith item in the stack; l and u denote the index range of the array in use; usually the values of these indices are 1 and SIZE respectively. TOP is a pointer to point the position of the array up to which it is filled with the items of the stack. With this representation, the following two ways can be stated:

EMPTY: TOP < 1

FULL: TOP ≥ u



(a) Array representation of a stack (b) Linked list representation of a stack

Figure 4.3 Two ways of representing stacks.

4.3.2 Linked List Representation of Stacks

Although array representation of stacks is very easy and convenient but it allows the representation of only fixed sized stacks. In several applications, the size of the stack may vary during program execution. An obvious solution to this problem is to represent a stack using a linked list. A single linked list structure is sufficient to represent any stack. Here, the DATA field is for the ITEM, and the LINK field is, as usual, to point to the next item. Figure 4.3(b) depicts such a stack using a single linked list.

In the linked list representation, the first node on the list is the current item that is the item at the top of the stack and the last node is the node containing the bottom-most item. Thus, a PUSH operation will add a new node in the front and a POP operation will remove a node from the front of the list. The SIZE of the stack is not important here because this representation allows dynamic stacks instead of static stacks, as with arrays.

In the linked list representation of a stack, whether a stack is empty or not can be ascertained by testing the LINK field of the STACK_HEAD node. Note that a test for overflow is not applicable in this case.

4.4 OPERATIONS ON STACKS

The basic operations required to manipulate a stack are:

PUSH	To insert an item into a stack
POP	To remove an item from a stack
STATUS	To know the present state of a stack

Let us define all these operations of a stack. First, we will consider the above-mentioned operations for a stack represented by an array.

Algorithm Push_Array
Input: The new item ITEM to be pushed onto it.
Output: A stack with a newly pushed ITEM at the TOP position.
Data structure: An array *A* with TOP as the pointer.

Steps:

1. **If** TOP ≥ SIZE **then**
2. **Print** "Stack is full"
3. **Else**
4. TOP = TOP + 1
5. A[TOP] = ITEM
6. **EndIf**
7. **Stop**

Here, we have assumed that the array index varies from 1 to SIZE and TOP points the location of the current top-most item in the stack. The following algorithm *Pop_Array* defines the POP of an item from a stack which is represented using an array A.

Algorithm Pop_Array
Input: A stack with elements.
Output: Removes an ITEM from the top of the stack if it is not empty.
Data structure: An array *A* with TOP as the pointer.

Steps:

1. **If** TOP < 1 **then**
2. **Print** "Stack is empty"
3. **Else**
4. ITEM = A[TOP]
5. TOP = TOP – 1
6. **EndIf**
7. **Stop**

In the following algorithm *Status_Array*, we test the various states of a stack such as whether it is full or empty, how many items are right now in it, and read the current element at the top without removing it, etc.

Algorithm Status_Array
Input: A stack with elements.
Output: States whether it is empty or full, available free space and item at TOP.
Data structure: An array *A* with TOP as the pointer.

Steps:

```
1.  If TOP < 1 then
2.      Print "Stack is empty"
3.  Else
4.      If (TOP ≥ SIZE) then
5.          Print "Stack is full"
6.      Else
7.          Print "The element at TOP is", A[TOP]
8.          free = (SIZE – TOP)/SIZE * 100
9.          Print "Percentage of free stack is", free
10.     EndIf
11. EndIf
12. Stop
```

Now let us see how the same operations can be defined for a stack represented with a single linked list.

Algorithm Push_LL
Input: ITEM is the item to be inserted.
Output: A single linked list with a newly inserted node with data content ITEM.
Data structure: A single linked list structure whose pointer to the header is known from STACK_HEAD and TOP is the pointer to the first node.

Steps:

```
1. new = GetNode(NODE)
   /* Insert at front */
2. new→DATA = ITEM
3. new→LINK = TOP
4. TOP = new
5. STACK_HEAD→LINK = TOP
6. Stop
```

Algorithm Pop_LL
Input: A stack with elements.
Output: The removed item is stored in ITEM.
Data structure: A single linked list structure whose pointer to the header is known from STACK_HEAD and TOP is the pointer to the first node.

Steps:

1. **If** TOP = NULL
2. **Print** "Stack is empty"
3. **Exit**
4. **Else**
5. ptr = TOP→LINK
6. ITEM = TOP→DATA
7. STACK_HEAD→LINK = ptr
8. TOP = ptr
9. **EndIf**
10. **Stop**

The operation *Status* now can be defined as follows:

Algorithm Status_LL( )

Input: A stack with elements.

Output: Status information such as its state (empty or full), number of items, item at the TOP.

Data structure: A single linked list structure whose pointer to the header is known from STACK_HEAD and TOP is the pointer to the first node.

Steps:

1. ptr = STACK_HEAD→LINK
2. **If** (ptr = NULL) **then**
3. **Print** "Stack is empty"
4. **Else**
5. nodeCount = 0
6. **While** (ptr ≠ NULL) **do**
7. nodeCount = nodeCount + 1
8. ptr = ptr→LINK
9. **EndWhile**
10. **Print** "The item at the front is", TOP→DATA, "Stack contains", nodeCount, "Number of items"
11. **EndIf**
12. **Stop**

Assignment 4.1

Using only PUSH, POP and STATUS as the basic operations defined in Section 4.4, write the procedures to solve the following problems.

(a) Read the top-most element in the stack. (Note that read will not remove the element.)

(b) Visit all elements in a stack. (Preferably without using another stack).

(c) Search for an item in the stack.

(Note: You should consider both types of memory representations of the stack.)

Assignment 4.2 (

In several applicatio
whereas others are
(Figure 4.4). One ca
N_y elements for sta
a single array, say
starting locations c
'grows' to the rig

With this schem
elements. This te
overflow even th
stacks.
Using this scher
PUSH_X, PUSH
defined to test t
Write the basic

4.5 APPLIC

Various applicati
evaluation of ar
arithmetic expres
use built-in stack
during the execu
recursive progra
memory variabl
known: the *stat*
is possible usin
The followi

4.5.1 Evalua

An arithmetic
constants and c
example, – (
(exponentiatio